

YES — you've hit one of the most confusing parts of Auth.js. 😊 There are **two different** `signIn`s, and then there's a bunch of behavior Auth.js gives you automatically.

Let's untangle it from the ground up.

1. These two `signIn`s are NOT the same thing

You have:

```
// auth.ts

export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({
  providers: [
    Google(...)
  ]
});
```

and:

```
import { signIn } from "next-auth/react";
```

They have the **same name**, but they are different functions for different environments.

`signIn` from `auth.ts`

This:

```
export const { signIn } = NextAuth(...)
```


is an **Auth.js server-side function**.

You might use it in server-side code, such as a Server Action.

```
"use server";

import { signIn } from "@auth";

export async function login() {
  await signIn("google");
}
```

It runs on the **server**.

signIn from "next-auth/react"

This:

```
import { signIn } from "next-auth/react";
```

is the **client-side React helper** provided by the **next-auth** package.

You use it in a Client Component:

```
"use client";

import { signIn } from "next-auth/react";

export default function LoginButton() {
  return (
    <button onClick={() => signIn("google")}>
      Continue with Google
    </button>
  );
}
```

It runs in the **browser** and initiates the Auth.js sign-in flow.

So:



```
@/auth
  ↓
signIn()
  ↓
SERVER version

next-auth/react
  ↓
signIn()
  ↓
CLIENT/BROWSER version
```

Same name. Different function.

Think of it like:

```
import { signIn } from "@auth";
```

means:

"Give me the `signIn` function that my `auth.ts` configured by calling `NextAuth()`."

Whereas:

```
import { signIn } from "next-auth/react";
```

means:

"Give me the **React/browser helper** that the `next-auth` package provides."

2. Now your bigger question: how do we get back to `/dashboard` ?!

This is the REALLY important part.

You asked:

Google redirects to `/api/auth/callback/google`, then `Auth.js` handles it. But I never wrote code saying "go to dashboard". So how does the user end up on the logged-in page?

There are actually **two things** happening:

A. Auth.js finishes the authentication

B. Auth.js redirects the browser somewhere

And Auth.js handles this for you.

Let's walk through the entire thing.

3. Full Google login flow

Suppose your app is:

```
https://myapp.com
```



and you have:

```
<button onClick={() => signIn("google")}>  
  Login  
</button>
```

Step 1 — User clicks

Browser executes:

```
signIn("google")
```

The Auth.js client helper initiates the sign-in flow.

Eventually the browser is sent to Google.

```
Browser  
  |  
  | Login with Google  
  ▼  
Google
```



4. User logs into Google

Google handles:

- email
- password
- MFA
- consent
- etc.

Your application **never sees the Google password**.

After authentication, Google has an authorization code:

ABC123

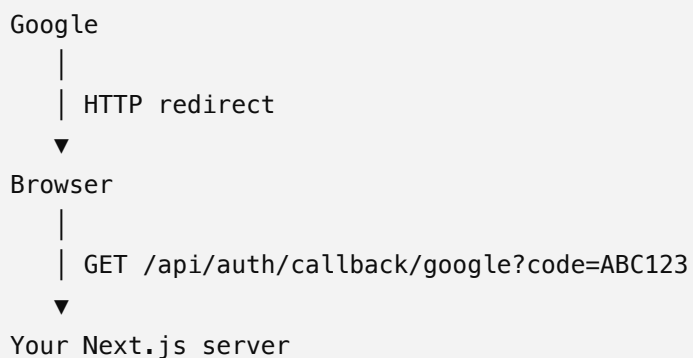


Google redirects the browser to the callback URL that you registered:

`https://myapp.com/api/auth/callback/google?code=ABC123`



So:



5. Next.js finds your catch-all route

You have:

```
app/  
└─ api/  
    └─ auth/  
        └─ [...nextauth]/  
            └─ route.ts
```



And:

```
import { handlers } from "@auth";  
  
export const { GET, POST } = handlers;
```

Next.js says:

"I have a GET request for `/api/auth/callback/google` . That matches `[...nextauth]` ."

Then:

```
Next.js  
  ↓  
GET handler  
  ↓  
Auth.js
```



6. Auth.js processes the callback

Auth.js receives:

```
/api/auth/callback/google?code=ABC123
```



Auth.js knows:

"This is a Google callback."

It takes the authorization code and communicates with Google **from the server**.

Conceptually:

```
Next.js/Auth.js  
  |  
  | code = ABC123  
  ▼  
Google token endpoint  
  |  
  ▼  
Google returns tokens / identity information
```



Auth.js can now determine the authenticated Google identity.

For example:

```
Google user:  
email = gopi@example.com
```



```
name = Gopi  
googleAccountId = 123456
```

7. Auth.js creates/finds the user

Now your adapter comes into play if you're using database-backed auth.

Conceptually:

```
Auth.js  
  ↓  
Adapter  
  ↓  
Prisma  
  ↓  
PostgreSQL
```



Auth.js asks the adapter something like:

```
"Do I already have a user connected to this Google account?"
```

If existing:

```
Google account  
  ↓  
existing User
```



If first login:

```
Google account  
  ↓  
create User  
  ↓  
create Account
```



8. Then Auth.js creates the session

This is where your logged-in state is established.

For a database-session setup:

```
Auth.js  
  ↓  
Adapter
```



↓
create Session

Database might contain:

Session



sessionToken
userId
expires

Then Auth.js sends a cookie to the browser.

Something conceptually like:

```
Set-Cookie:  
  authjs.session-token=ABCXYZ;  
  HttpOnly;  
  Secure;  
  SameSite=Lax
```

So the browser now has:

Cookie
authjs.session-token = ABCXYZ



🎉 **At this point, the browser is logged in.**

But we're not necessarily at `/dashboard` yet.

9. Now Auth.js redirects the browser

This is the part you were asking about.

Auth.js sends an HTTP redirect response.

Conceptually:

```
HTTP/1.1 302 Found  
Location: /dashboard  
Set-Cookie: authjs.session-token=ABCXYZ; ...
```

The browser sees:

Location: /dashboard



and automatically navigates there.

You didn't write:

```
router.push("/dashboard")
```

because **Auth.js** generated the redirect response.

So:

```
Google
  ↓
/api/auth/callback/google
  ↓
Auth.js
  ↓
create session
  ↓
Set-Cookie
  ↓
HTTP redirect
  ↓
/dashboard
```



10. But WAIT — how does Auth.js know /dashboard ?!

Excellent question. 😊

There are a few possibilities depending on how you initiate/configure the flow.

For example, the client helper can accept a callback URL:

```
signIn("google", {
  callbackUrl: "/dashboard"
});
```

Conceptually:


```
signIn()
```

↓

```
"After authentication, eventually return me to /dashboard"
```



Auth.js carries that information through the authentication flow and uses it when finishing the callback.

In modern Auth.js versions, there are also redirect/callback configuration mechanisms and defaults.

So the important point is:

The redirect is performed by Auth.js; your application doesn't have to manually handle the Google callback request.

11. Now where does `/api/auth/session` come in?

This is another thing that gets mixed up with the callback.

The callback establishes the login.

The session endpoint lets the client ask, "What's my current session?"

They're different jobs.

Think:

```
CALLBACK
```

```
"Google login just finished.  
Create/establish my session."
```

↓

```
SESSION
```

```
"Hey Auth.js,  
what is my current logged-in session?"
```



12. Example: Client Component wants to know who is logged in

Suppose you have:

```
"use client";

import { useSession } from "next-auth/react";

export function Profile() {
  const { data: session } = useSession();

  return <div>Hello {session?.user?.name}</div>;
}
```

`useSession()` needs session information.

Auth.js's React client machinery can make a request to:

`/api/auth/session`



Conceptually:

```
Browser
  |
  | GET /api/auth/session
  | Cookie: authjs.session-token=ABCXYZ
  ▼
Next.js
  ↓
Auth.js
  ↓
session lookup/verification
  ↓
return session information
  ↓
Browser
```



Response could conceptually look like:

```
{
  "user": {
    "name": "Gopi",
    "email": "gopi@example.com"
  },
  "expires": "...
}
```

Then React knows:

```
session = {  
  user: {  
    name: "Gopi",  
    email: "gopi@example.com"  
  }  
}
```



and can render:

Hello Gopi 🙌



13. So callback vs session

This distinction is **very important** for interviews.

Callback

```
/api/auth/callback/google
```



means:

"Google has finished authenticating me. Process the OAuth callback and establish the Auth.js login/session."

It happens **during the login flow**.

Session

```
/api/auth/session
```



means:

"What is my current Auth.js session?"

It can be requested **after login** whenever client-side code needs session information.

14. And here's a VERY important Next.js distinction

If you're in a **Server Component**, you often don't need:

/api/auth/session



at all.

You can do:

```
import { auth } from "@auth";

export default async function Dashboard() {
  const session = await auth();

  if (!session) {
    // redirect
  }

  return <h1>Hello {session.user?.name}</h1>;
}
```

Here:

Browser

↓

GET /dashboard

Cookie: authjs.session-token=ABCXYZ

▼

Next.js server

↓

Dashboard Server Component

↓

auth()

↓

Auth.js reads/resolves session

↓

session

↓

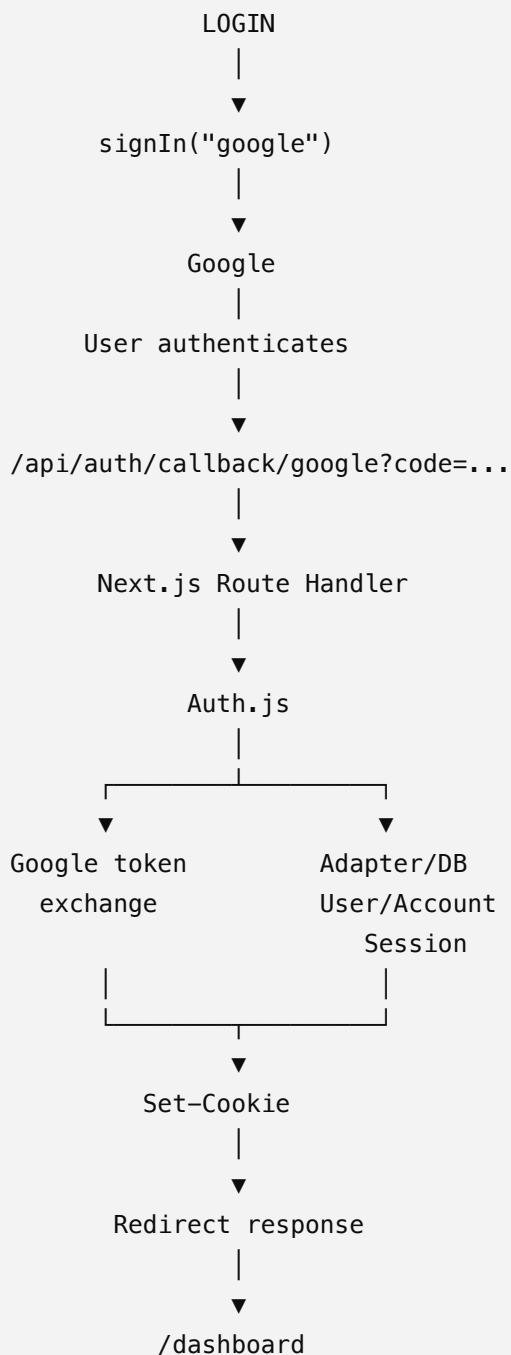
render dashboard



No browser request to `/api/auth/session` is necessary for this particular server-side check.

15. Put the whole thing together 🧠

Here's the mental model I'd memorize:



Then later:

Server Component



↓
render dashboard

Client Component using `useSession()`

Client Component
↓
`useSession()`
↓
`/api/auth/session`
↓
`Auth.js`
↓
`session`
↓
React UI



🔥 One final distinction

Don't think:

`/api/auth/session` = the thing that makes me logged in.

Instead:

Callback
= establishes the login/session

Cookie
= carries the session credential on requests

Session lookup
= tells the app what that session represents

`/api/auth/session`
= one way client-side code can ask `Auth.js` for that information



And `auth()` is the server-side equivalent you commonly use in Next.js Server Components/server code.

That separation — **callback** → **establish session** → **cookie** → **later session lookup** — is the piece that makes the whole `Auth.js` architecture click. 🧩



import { auth } from "@auth";..is this default or written code somewhere ? and And the really interesting next question is: **when** `/api/auth/callback/google` **arrives** with `?code=ABC123 ****`, **how does Auth.js know which Google account/user this code belongs to and then create the session?** That's the heart of the OAuth flow.

Exactly. 😊 Let's answer both, because they connect directly.

1. Where does `import { auth } from "@auth"` come from?

You wrote the `auth.ts` file. But the `auth` function itself is **generated/provided** by **Auth.js** when you call `NextAuth()`.

Your code:

</> TypeScript



// auth.ts

```
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";

export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({
  providers: [
    Google({
      clientId: process.env.AUTH_GOOGLE_ID,
      clientSecret: process.env.AUTH_GOOGLE_SECRET,
    }),
  ],
});
```

So this part:

</> TypeScript



```
auth
```

comes out of:

```
</> TypeScript  
NextAuth({...})
```



Then you export it:

```
</> TypeScript  
export const { auth } = NextAuth({...});
```



And somewhere else:

```
</> TypeScript  
import { auth } from "@auth";
```



means:

"Import the `auth` function that my `auth.ts` file exported."

So the chain is:

```
next-auth package  
  ↓  
  NextAuth()  
  ↓  
  returns { handlers, auth, signIn, signOut }  
  ↓  
  your auth.ts exports them  
  ↓  
  other files import { auth } from "@auth"
```



`@/auth` is **not** a package.

It's your own:

```
auth.ts
```



file, with `@/` usually being your project's root path alias.

2. Now the BIG OAuth question 🔥

You asked:

When `/api/auth/callback/google?code=ABC123` arrives, how does Auth.js know which Google account/user this code belongs to?

The answer is:

The authorization code itself is a temporary reference to the Google authorization transaction.

Your app does **not** look at:

ABC123



and somehow decode:

Gopi
gopi@gmail.com
Google user ID 12345



✗ That's not how it works.

The code is basically a **short-lived, one-time credential/reference** that Google can **recognize**.

Let's follow it properly.

3. Start before the callback

You click:

```
</> TypeScript  
  
signIn("google")
```



Auth.js knows you configured Google:

```
</> TypeScript  
  
providers: [  
  Google(...)  
]
```



So Auth.js constructs an OAuth authorization URL roughly like:

```
https://accounts.google.com/o/oauth2/v2/auth  
?client_id=YOUR_CLIENT_ID
```



```
&redirect_uri=https://myapp.com/api/auth/callback/google
&response_type=code
&scope=openid profile email
&state=XYZ
```

Don't worry about every parameter yet.

The important ones are:

```
client_id
redirect_uri
response_type=code
scope
state
```



4. Browser goes to Google

```
Browser
  |
  | GET Google authorization URL
  ▼
Google
```



Google now knows:

"This request is coming from the application registered as `YOUR_CLIENT_ID`."

Google shows its login/consent screen.

5. User authenticates with Google

Suppose you log in as:

```
gopi@gmail.com
```



Google now knows:

```
Google user
  ↓
gopi@gmail.com
```



Google also knows:

This user authorized YOUR application



Google creates a temporary authorization code:

ABC123



Conceptually:

Google's internal state



ABC123

↓

this authorization transaction

↓

Google account: 123456789

↓

email: gopi@gmail.com

↓

client: YOUR_CLIENT_ID

↓

redirect URI: https://myapp.com/api/auth/callback/google

↓

expires soon

The exact internal representation is Google's implementation detail.

Your application **doesn't need to know what's encoded inside the code.**

6. Google redirects the browser

Google tells the browser:

</> http



302 Found

Location: https://myapp.com/api/auth/callback/google?code=ABC123&state=XYZ

Browser follows it:

Browser



|

| GET /api/auth/callback/google?code=ABC123

▼

Your Next.js server

Now Auth.js receives:

`code = ABC123`

7. Does Auth.js know the user yet?

Not really.

This is the crucial part.

Auth.js basically says:

"Cool. I have an authorization code. I need to take this to Google and exchange it for tokens."

Auth.js makes a **server-to-server HTTPS request to Google's token endpoint**.

Conceptually:

```
Your server/Auth.js
  |
  | POST
  | code=ABC123
  | client_id=...
  | client_secret=...
  ▼
Google token endpoint
```



Notice something important:

The `client_secret` is sent from your server.

That's why your Google client secret must NOT be in browser code.

8. Google validates the code

Google checks things such as:

- Is the code valid?
- Has it expired?
- Has it already been used?
- Was it issued to this client?
- Does the redirect/client information match the authorization transaction?

If everything is valid, Google returns tokens.

Conceptually:

```
</> JSON

{
  "access_token": "...",
  "id_token": "...",
  "expires_in": 3600
}
```



For OIDC, the important one for **identity** is the:

id_token



9. What's inside the ID token?

This is where we finally get the user's identity.

An ID token is a JWT containing claims.

Conceptually:

```
</> JSON

{
  "iss": "https://accounts.google.com",
  "sub": "123456789",
  "aud": "YOUR_CLIENT_ID",
  "email": "gopi@gmail.com",
  "name": "Gopi Chandana"
}
```



The exact claims can vary.

The important identity identifier is generally:

sub



Think:

```
Google
  ↓
sub = "123456789"
```



That is Google's stable identifier for that user within the relevant OIDC context.

So now Auth.js can know:

```
Google account
  ↓
provider = google
providerAccountId = 123456789
  ↓
email = gopi@gmail.com
```



10. NOW the database comes into the picture

Suppose you're using:

```
</> TypeScript

adapter: PrismaAdapter(prisma)
```



Auth.js can ask the adapter:

```
"Do I already have a Google account with provider google and providerAccountId 123456789 ?"
```

Conceptually:

```
Auth.js
  ↓
Adapter
  ↓
Prisma
  ↓
PostgreSQL
```



Your database might have:

User

id	name	email
U001	Gopi	<u>gopi@gmail.com ↗</u>

Account

userId	provider	providerAccountId
U001	google	123456789

That **Account** record is extremely important.

It creates the relationship:

```
Google account
  |
  | providerAccountId = 123456789
  ▼
Your application's User U001
```



11. First login vs returning login

First time Gopi logs in

Auth.js doesn't find:

```
google + 123456789
```



So it can create:

```
User
  ↓
U001 / gopi@gmail.com

Account
  ↓
google / 123456789
```



Then create the session.

Next time Gopi logs in

Google again identifies the same Google account:

```
providerAccountId = 123456789
```



Auth.js looks in the Account table:

```
google + 123456789
  ↓
  U001
```



It knows:

"Ah, this Google account belongs to my application's User U001."

It doesn't create another user.

12. Then Auth.js creates the session

Now:

Google identity

↓

Account

↓

User

↓

Session



For database sessions, conceptually:

Session

sessionToken = XYZABC

userId = U001

expires = ...



Then Auth.js sends a cookie:

</> http

Set-Cookie:

authjs.session-token=XYZABC;

HttpOnly;

Secure;

SameSite=Lax



Browser stores it.



Now the browser has the credential it will present on future requests.

13. And THEN comes your /dashboard

Auth.js can redirect the browser:

</> http



Location: /dashboard

Browser:

`</> http`

GET /dashboard

Cookie: authjs.session-token=XYZABC

Your Server Component:

`</> TypeScript`

```
import { auth } from "@auth";

export default async function Dashboard() {
  const session = await auth();

  if (!session) {
    // redirect to login
  }

  return <h1>Hello {session.user?.name}</h1>;
}
```

And `auth()` can resolve:

Cookie

↓

sessionToken

↓

Session

↓

userId

↓

User

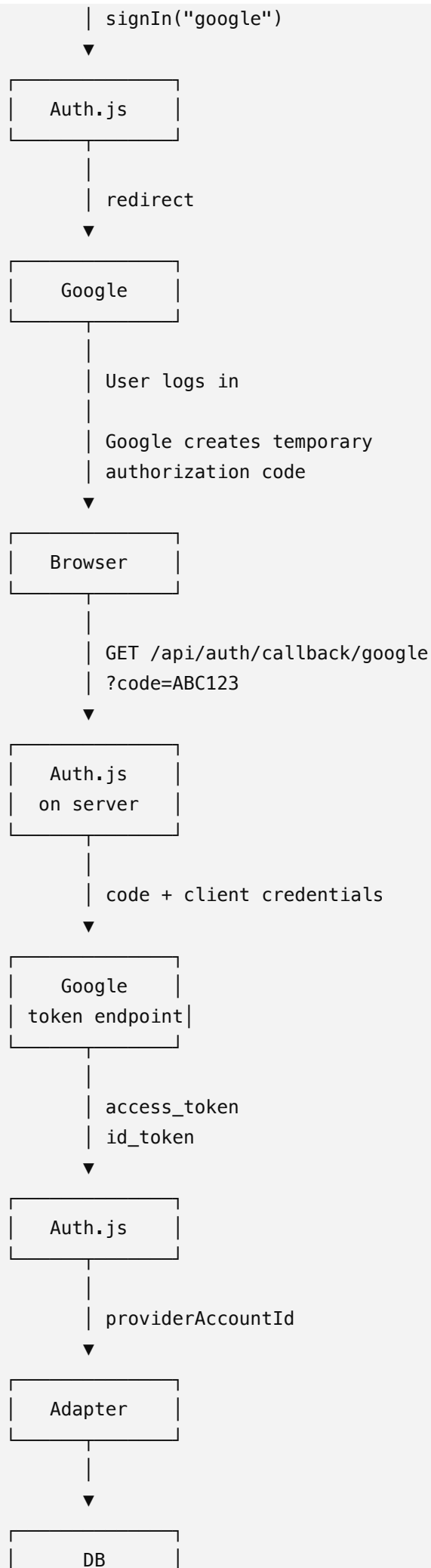
↓

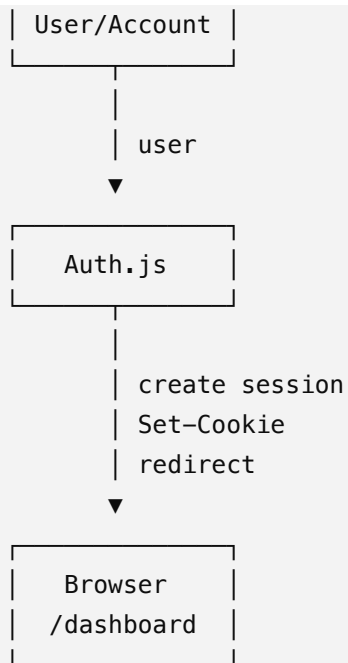
session object

The entire OAuth/OIDC flow

This is the diagram I want you to have in your head:

Browser





And the most important conceptual distinction:

OAuth

authorization code



"Google has authorized this application"

OIDC

ID token



"Here is the identity of the authenticated user"

Auth.js

takes that identity



connects it to your application's User



creates/manages your app session

Adapter

talks to your DB

Cookie

lets browser present the session on future requests

So **Auth.js** does **NOT** determine the Google user by decoding `ABC123` itself. Google recognizes the code when Auth.js exchanges it at Google's token endpoint, and the

resulting OIDC identity gives Auth.js the provider/account identity it can map to your application's user. That's the key OAuth flow. 🔥



Awesome 🤩 Let's move to the **next important Auth.js concept**: how the session is actually used to protect pages and APIs.

Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

```
Google
  ↓
OAuth/OIDC
  ↓
Auth.js callback
  ↓
User/Account
  ↓
Session
  ↓
Cookie
```



Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

1. Protecting a Server Component

Suppose we have:

```
app/
├─ login/
│   └─ page.tsx
└─ dashboard/
    └─ page.tsx
```



Our dashboard should only be accessible to logged-in users.

We can write:

```
</> TypeScript

// app/dashboard/page.tsx

import { auth } from "@auth";
```



```
import { redirect } from "next/navigation";

export default async function DashboardPage() {
  const session = await auth();

  if (!session) {
    redirect("/login");
  }

  return (
    <div>
      <h1>Welcome {session.user?.name}</h1>
      <p>{session.user?.email}</p>
    </div>
  );
}
```

Now let's understand **exactly what happens**.

2. User requests `/dashboard`

Browser:

```
</> http

GET /dashboard
Cookie: authjs.session-token=ABC123
```



Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript

const session = await auth();
```



runs.

3. What does `auth()` do?

Remember where it came from:

```
</> TypeScript
```




```
// auth.ts

export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({...});
```

So `auth()` is an Auth.js server-side function.

Conceptually:

```
graph TD
    Browser -- "Cookie: session-token=ABC123" --> Nextjs[Next.js]
    Nextjs --> Dashboard[Dashboard Server Component]
    Dashboard -- "auth()" --> Authjs[Auth.js]
    Authjs --> Session[Session verification / lookup]
    Session --> UserSession[User session]
```

If the session is valid:

```
</> TypeScript

session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

```
</> TypeScript
```

```
const session = await auth();
```



returns something equivalent to:

```
</> TypeScript
```

```
null
```



So:

```
</> TypeScript
```

```
if (!session) {  
  redirect("/login");  
}
```



executes.

Next.js sends a redirect response.

Conceptually:

```
</> http
```

```
307 Temporary Redirect  
Location: /login
```



The browser then requests:

```
</> http
```

```
GET /login
```



So:

```
User → /dashboard
```

```
↓
```

```
auth()
```

```
↓
```

```
no session
```

```
↓
```

```
redirect("/login")
```



↓
/login

5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript  
  
if (!session)
```



That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

```
</> TypeScript  
  
const session = await auth();  
  
if (!session) {  
  redirect("/login");  
}  
  
if (session.user.role !== "ADMIN") {  
  redirect("/forbidden");  
}
```



Now we're asking two different questions:

Authentication:
"Who are you / are you logged in?"



```
Authorization:
"Are you allowed to do this?"
```

6. But there's an important security rule

Never rely only on hiding UI.

For example:

```
</> TypeScript

{session?.user.role === "ADMIN" && (
  <button>Delete User</button>
)}
```

This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http

POST /api/users/delete
```

Therefore your server-side action/API must check authorization too.

7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```

We can do:

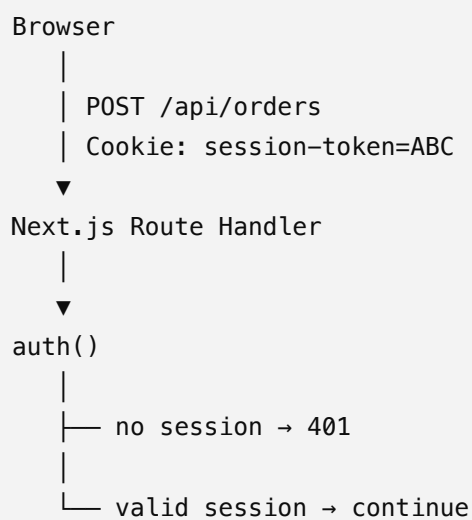
```
</> TypeScript

import { auth } from "@auth";
import { NextResponse } from "next/server";

export async function POST(request: Request) {
  const session = await auth();
```

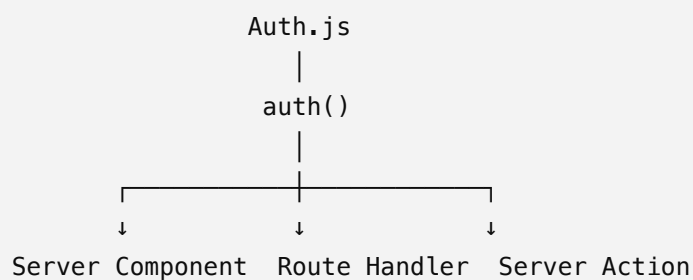
```
if (!session) {  
  return NextResponse.json(  
    { error: "Unauthorized" },  
    { status: 401 }  
  );  
}  
  
// User is authenticated  
// Continue processing order...  
  
return NextResponse.json({  
  success: true,  
});  
}
```

Now:



8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

Server Component

</> TypeScript



```
const session = await auth();
```

Route Handler

</> TypeScript



```
const session = await auth();
```

Server Action

</> TypeScript



```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

Your server-side code asks Auth.js to resolve the current authenticated user/session.

9. Server Action example

Suppose:

</> TypeScript



```
"use server";

import { auth } from "@auth";

export async function deleteAccount() {
  const session = await auth();

  if (!session) {
    throw new Error("Unauthorized");
  }

  // Delete account...
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

10. Interview answer

If an interviewer asks:

"How would you protect a page in Next.js using Auth.js?"

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

One thing I want you to notice

We now have **three different places** where Auth.js can participate:

```
1. LOGIN
  signIn()
    ↓
  OAuth
    ↓
  callback
    ↓
  session created

2. READING SESSION
  auth()
    ↓
  "Who is currently logged in?"

3. PROTECTING ACTIONS
  auth()
    ↓
```



```
authenticate
  ↓
authorize
  ↓
perform operation
```

That distinction is **very interview-important**.

Next after this

The natural next piece is **Auth.js callbacks** — specifically:

</> TypeScript

```
callbacks: {
  async jwt() {},
  async session() {},
  async signIn() {},
  async redirect() {}
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥

📄 👍 ⬆️ ↺ ...

how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

🔒 Remaining Auth.js / Auth & Authorization lessons

#	Topic	What you'll learn
7.6	🛡️ Route Protection	<code>auth()</code> , protecting pages, APIs, Server Actions

#	Topic	What you'll learn
7.7	 Auth.js Callbacks	<code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks
7.8	 Session Customization	Put <code>user.id</code> , <code>role</code> , etc. into the session
7.9	 Authorization / RBAC	Roles, permissions, admin/user access
7.10	 Session Strategies	Database sessions vs JWT sessions
7.11	 CSRF + SameSite	Why state-changing requests need protection
7.12	 Middleware / Proxy Auth	Protecting groups of routes before rendering
7.13	 Logout & Session Expiry	Sign-out, expiration, invalid/expired sessions
7.14	 Complete Auth Architecture	Put Browser → Next.js → Auth.js → OAuth → DB → Session together
7.15	 Auth Interview Questions	Real interview scenarios + architecture questions

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

1. Callbacks
2. Session customization
3. Authorization/RBAC
4. JWT vs DB sessions
5. CSRF/SameSite
6. Middleware/Proxy

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 🍌 After we finish those remaining Auth.js topics, we can consider the **Authentication & Authorization** section complete.

More precisely, your **Next.js Auth & Authorization roadmap** will cover:

AUTHENTICATION



- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT
- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

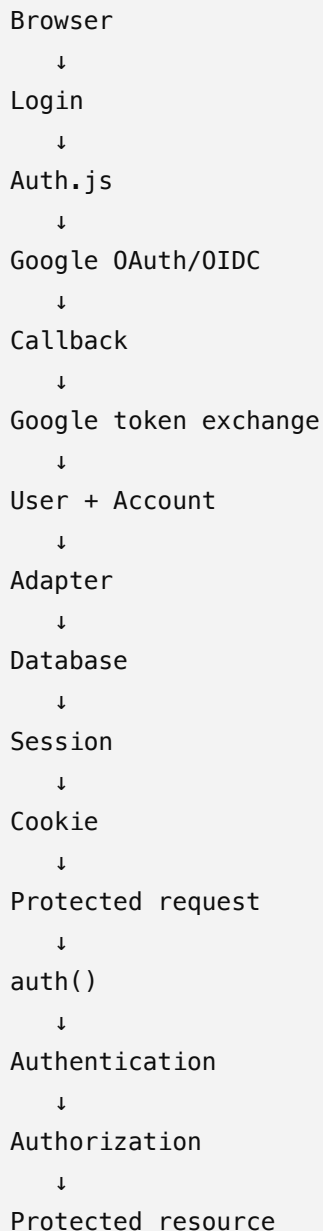
AUTHORIZATION

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

SECURITY

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:



```
graph TD; Browser --> Login; Login --> Authjs[Auth.js]; Authjs --> Google[Google OAuth/OIDC]; Google --> Callback; Callback --> GoogleToken[Google token exchange]; GoogleToken --> User[User + Account]; User --> Adapter; Adapter --> Database; Database --> Session; Session --> Cookie; Cookie --> ProtectedRequest[Protected request]; ProtectedRequest --> auth[auth()]; auth --> Authentication; Authentication --> Authorization; Authorization --> ProtectedResource[Protected resource];
```

Browser

↓

Login

↓

Auth.js

↓

Google OAuth/OIDC

↓

Callback

↓

Google token exchange

↓

User + Account

↓

Adapter

↓

Database

↓

Session

↓

Cookie

↓

Protected request

↓

auth()

↓

Authentication

↓

Authorization

↓

Protected resource

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE. 🗝️✅



ChatGPT can make mistakes. Check important info.